

資料結構

Data Structure

Lab 09

姓名： 黃聖傑

學號： 114310140

Lab09-Q1.

Extend the Example by Adding Deletion Functionality to the Binary Search Tree.

Code

```
#include <iostream>
using namespace std;

// =====
// 樹的節點類別
// =====
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點
    TreeNode* right;    // 指向右子節點

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// =====
// 二元搜尋樹類別
// =====
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }

        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
        }
    }
};
```

```

    }
    // 如果 val == node->data，不插入（避免重複）

    return node; // 回傳更新後的節點
}

// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
    if (node == nullptr) return false;

    if (node->data == target) return true;
    else if (target < node->data) return search(node->left, target); // 繼續
在左子樹找
    else return search(node->right, target); // 繼續在右子樹找
}

// --- 私有的中序遍歷方法（左-根-右） ---
void inOrderTraversal(TreeNode* node) {
    if (node == nullptr) return;

    inOrderTraversal(node->left); // 先走訪左子樹
    cout << node->data << " "; // 印出目前節點的資料
    inOrderTraversal(node->right); // 再走訪右子樹
}

// --- 私有的刪除方法 ---
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) {
        cout << "值 " << val << " 不存在，無法刪除。" << endl;
        return nullptr; // 沒有找到要刪除的節點
    }

    if (val < node->data) {
        node->left = deleteNode(node->left, val); // 在左子樹刪除
    }
    else if (val > node->data) {
        node->right = deleteNode(node->right, val); // 在右子樹刪除
    }
}

```

```

else{
    // 找到要刪除的節點
    if (node->left == nullptr) {
        TreeNode* temp = node->right; // 只有右子樹
        delete node; // 刪除當前節點
        return temp; // 回傳新的子樹根節點
    }
    else if (node->right == nullptr) {
        TreeNode* temp = node->left; // 只有左子樹
        delete node; // 刪除當前節點
        return temp; // 回傳新的子樹根節點
    }
    else {
        // 有兩個子樹，找到右子樹的最小值替代當前節點
        TreeNode* temp = findMin(node->right);
        node->data = temp->data; // 替換資料
        node->right = deleteNode(node->right, temp->data); // 刪除右子樹中
        的最小值節點

        return node; // 回傳更新後的節點
    }
}

return node; // 回傳更新後的節點
}

// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) {
        node = node->left;
    }
    return node;
}

public:
    // --- 建構子：初始化為空樹 ---
    BinarySearchTree() : root(nullptr) {}

    // --- 公開插入方法 ---

```

```

void insert(int val) {
    root = insert(root, val);
}

// --- 公開搜尋方法 ---
bool search(int target) {
    return search(root, target);
}

// --- 公開中序遍歷方法 ---
void displayInOrder() {
    cout << "樹的中序遍歷（排序後）結果：" << endl;
    inOrderTraversal(root);
    cout << endl;
}

// --- 公開刪除方法 ---
void deleteValue(int val) {
    root = deleteNode(root, val);
}
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料
    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]);

    // 把資料插入樹中
    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }

    // 讓使用者輸入要插入的鍵值
    int key;
    cout << "請輸入要搜尋/新增的鍵值：";
    cin >> key;
}

```

```
    if (bst.search(key)) {
        cout << "值 " << key << " 已存在樹中，不新增。" << endl;
    }
    else {
        cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
        bst.insert(key);
    }

    // 顯示目前樹的中序遍歷結果
    bst.displayInOrder();

    // 讓使用者輸入要刪除的鍵值
    int deleteKey;
    cout << "請輸入要刪除的鍵值：" << endl;
    cin >> deleteKey;

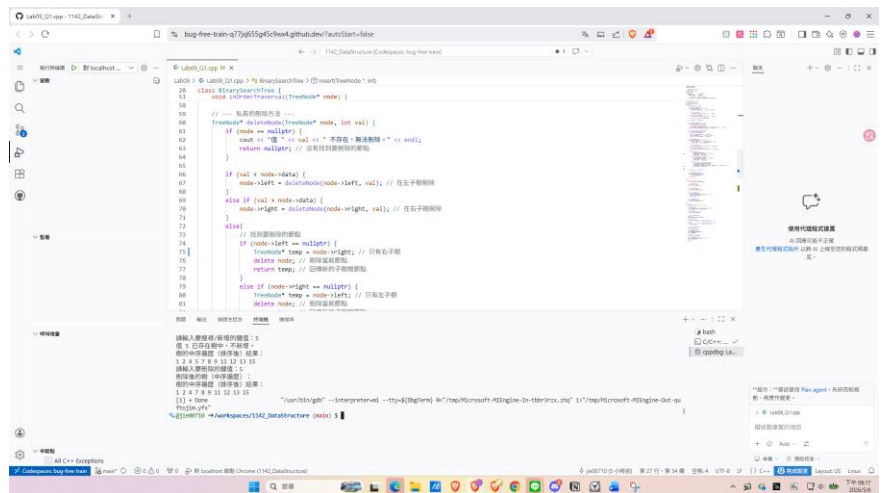
    // 刪除節點
    bst.deleteValue(deleteKey);

    // 顯示刪除後的樹
    cout << "刪除後的樹（中序遍歷）：" << endl;
    bst.displayInOrder();

    return 0;
}
```

Result

(a)

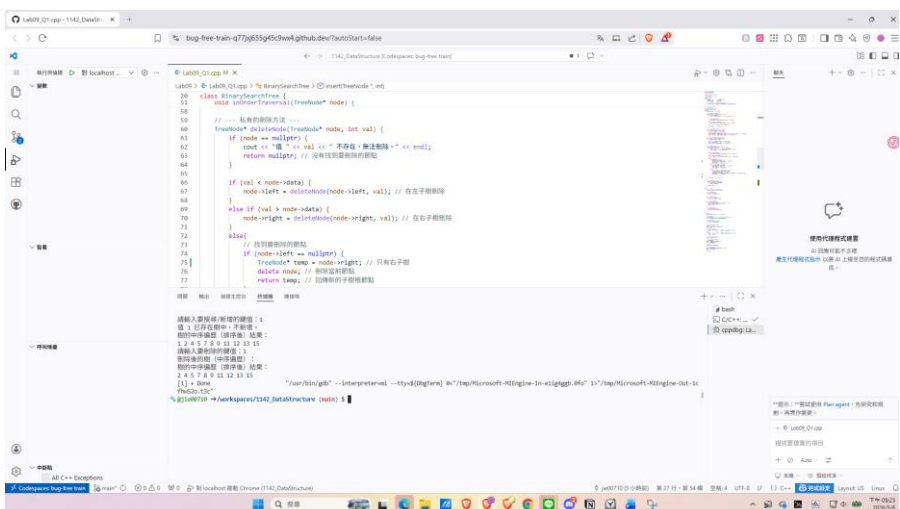


```
1 // 1142_DataStructure\Codepoints\bug-free-tree\
2
3 #include <iostream>
4 using namespace std;
5
6 // 二叉树的节点定义
7 struct TreeNode {
8     int val;
9     TreeNode* left;
10    TreeNode* right;
11    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
12 };
13
14 // 插入节点的方法
15 void insert(TreeNode*& node, int val) {
16     if (node == NULL) {
17         cout << "插入失败: 节点已存在" << endl;
18         return;
19     }
20     if (val < node->val) {
21         insert(node->left, val);
22     } else if (val > node->val) {
23         insert(node->right, val);
24     }
25 }
26
27 // 删除节点的方法
28 void deleteNode(TreeNode*& node, int val) {
29     if (node == NULL) {
30         cout << "删除失败: 节点不存在" << endl;
31         return;
32     }
33     if (val < node->val) {
34         deleteNode(node->left, val);
35     } else if (val > node->val) {
36         deleteNode(node->right, val);
37     } else {
38         // 找到要删除的节点
39         if (node->left == NULL) {
40             delete node;
41             return;
42         } else if (node->right == NULL) {
43             delete node;
44             return;
45         }
46         // 左右子树都不为空
47         int temp = node->right->val;
48         deleteNode(node->right, temp);
49         node->val = temp;
50     }
51 }
52
53 // 打印二叉树的方法
54 void print(TreeNode* node) {
55     if (node == NULL) return;
56     print(node->left);
57     cout << node->val << " ";
58     print(node->right);
59 }
60
61 int main() {
62     TreeNode* root = NULL;
63     insert(root, 5);
64     insert(root, 3);
65     insert(root, 7);
66     insert(root, 2);
67     insert(root, 4);
68     insert(root, 6);
69     insert(root, 1);
70     print(root);
71     deleteNode(root, 3);
72     print(root);
73     return 0;
74 }
```

控制台输出:

```
5 3 7 2 4 6 1
5 2 4 6 7 1
```

(b)

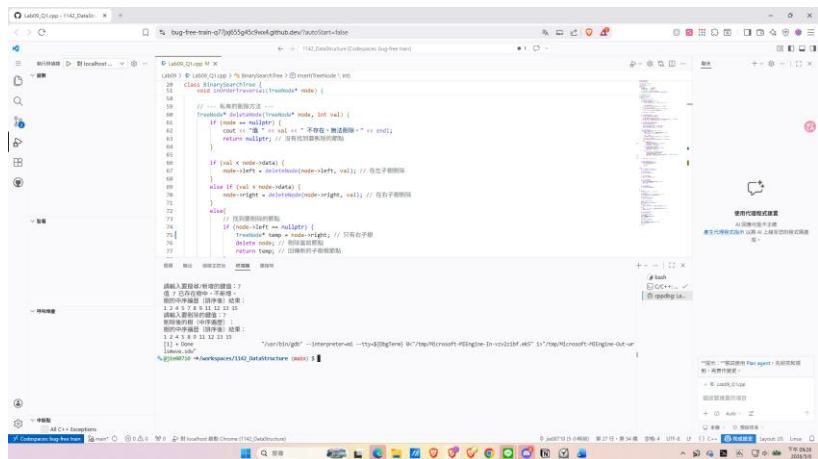


```
1 // 1142_DataStructure\Codepoints\bug-free-tree\
2
3 #include <iostream>
4 using namespace std;
5
6 // 二叉树的节点定义
7 struct TreeNode {
8     int val;
9     TreeNode* left;
10    TreeNode* right;
11    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
12 };
13
14 // 插入节点的方法
15 void insert(TreeNode*& node, int val) {
16     if (node == NULL) {
17         cout << "插入失败: 节点已存在" << endl;
18         return;
19     }
20     if (val < node->val) {
21         insert(node->left, val);
22     } else if (val > node->val) {
23         insert(node->right, val);
24     }
25 }
26
27 // 删除节点的方法
28 void deleteNode(TreeNode*& node, int val) {
29     if (node == NULL) {
30         cout << "删除失败: 节点不存在" << endl;
31         return;
32     }
33     if (val < node->val) {
34         deleteNode(node->left, val);
35     } else if (val > node->val) {
36         deleteNode(node->right, val);
37     } else {
38         // 找到要删除的节点
39         if (node->left == NULL) {
40             delete node;
41             return;
42         } else if (node->right == NULL) {
43             delete node;
44             return;
45         }
46         // 左右子树都不为空
47         int temp = node->right->val;
48         deleteNode(node->right, temp);
49         node->val = temp;
50     }
51 }
52
53 // 打印二叉树的方法
54 void print(TreeNode* node) {
55     if (node == NULL) return;
56     print(node->left);
57     cout << node->val << " ";
58     print(node->right);
59 }
60
61 int main() {
62     TreeNode* root = NULL;
63     insert(root, 5);
64     insert(root, 3);
65     insert(root, 7);
66     insert(root, 2);
67     insert(root, 4);
68     insert(root, 6);
69     insert(root, 1);
70     print(root);
71     deleteNode(root, 3);
72     print(root);
73     return 0;
74 }
```

控制台输出:

```
5 3 7 2 4 6 1
5 2 4 6 7 1
```

(c)



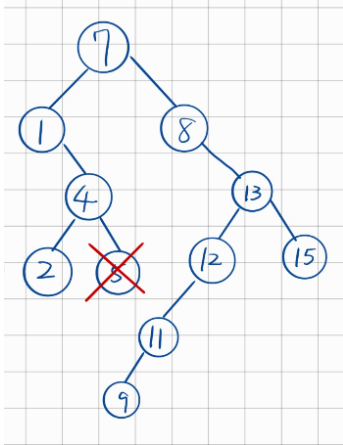
```
1 // 1142_DataStructure\Codepoints\bug-free-tree\
2
3 #include <iostream>
4 using namespace std;
5
6 // 二叉树的节点定义
7 struct TreeNode {
8     int val;
9     TreeNode* left;
10    TreeNode* right;
11    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
12 };
13
14 // 插入节点的方法
15 void insert(TreeNode*& node, int val) {
16     if (node == NULL) {
17         cout << "插入失败: 节点已存在" << endl;
18         return;
19     }
20     if (val < node->val) {
21         insert(node->left, val);
22     } else if (val > node->val) {
23         insert(node->right, val);
24     }
25 }
26
27 // 删除节点的方法
28 void deleteNode(TreeNode*& node, int val) {
29     if (node == NULL) {
30         cout << "删除失败: 节点不存在" << endl;
31         return;
32     }
33     if (val < node->val) {
34         deleteNode(node->left, val);
35     } else if (val > node->val) {
36         deleteNode(node->right, val);
37     } else {
38         // 找到要删除的节点
39         if (node->left == NULL) {
40             delete node;
41             return;
42         } else if (node->right == NULL) {
43             delete node;
44             return;
45         }
46         // 左右子树都不为空
47         int temp = node->right->val;
48         deleteNode(node->right, temp);
49         node->val = temp;
50     }
51 }
52
53 // 打印二叉树的方法
54 void print(TreeNode* node) {
55     if (node == NULL) return;
56     print(node->left);
57     cout << node->val << " ";
58     print(node->right);
59 }
60
61 int main() {
62     TreeNode* root = NULL;
63     insert(root, 5);
64     insert(root, 3);
65     insert(root, 7);
66     insert(root, 2);
67     insert(root, 4);
68     insert(root, 6);
69     insert(root, 1);
70     print(root);
71     deleteNode(root, 3);
72     print(root);
73     return 0;
74 }
```

控制台输出:

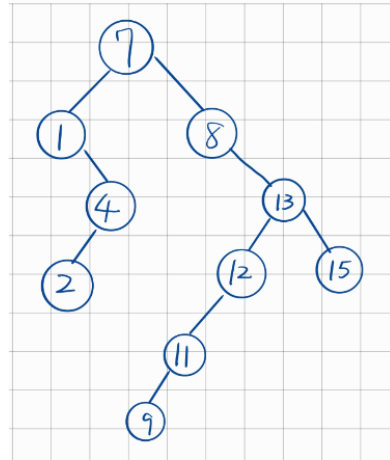
```
5 3 7 2 4 6 1
5 2 4 6 7 1
```

Discussion

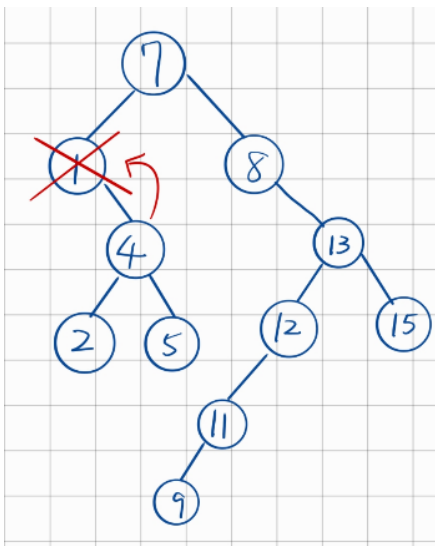
(a1)



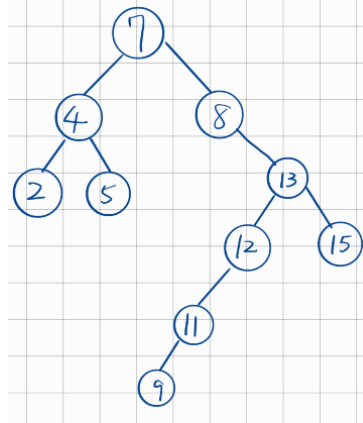
(a2)



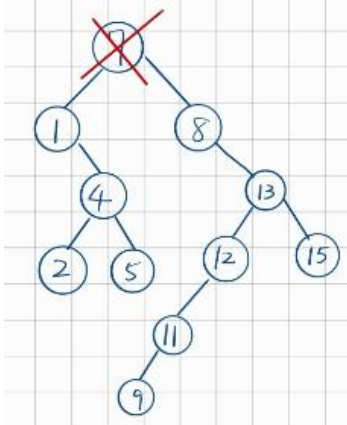
(b1)



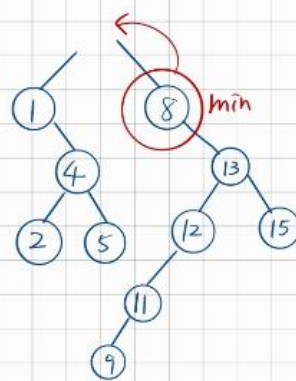
(b2)



(c1)



(c2)



(c3)

